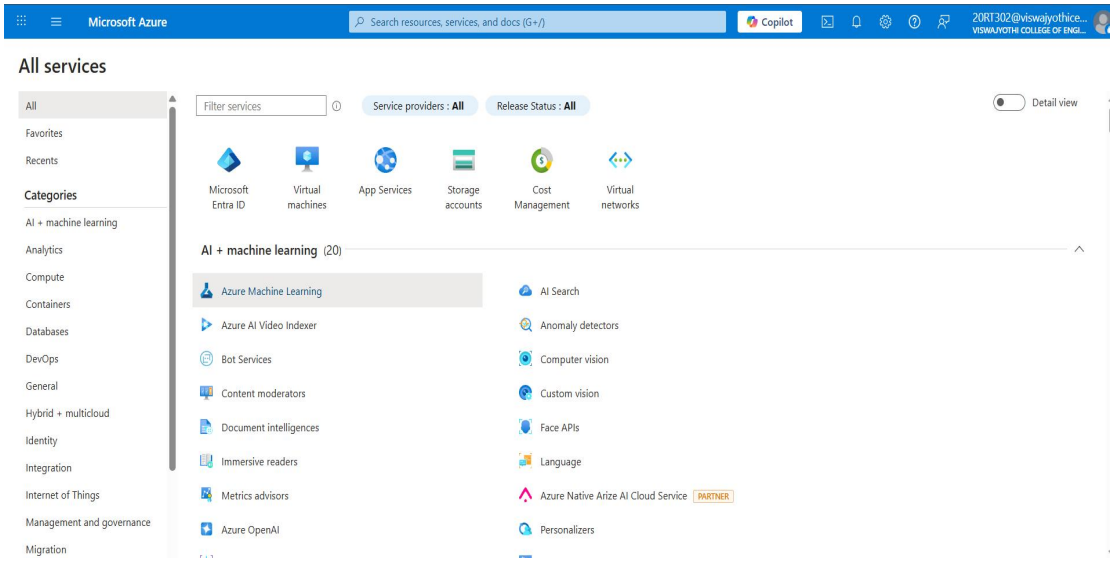
Building a Real-Time Weather Data Pipeline using Azure Services and Power BI

1. Project Overview

This project was my dive into the Azure cloud ecosystem. I wanted to see how a real-time data pipeline works from start to finish. The goal was to fetch live weather data for a few cities, store it in the cloud, integrate it into a database, and visualize it in Power BI.

The pipeline mimics enterprise data workflows, focusing on ingestion, storage, and visualization. While my dataset was small, the architecture is fully functional and could easily scale.

Azure Portal Services:



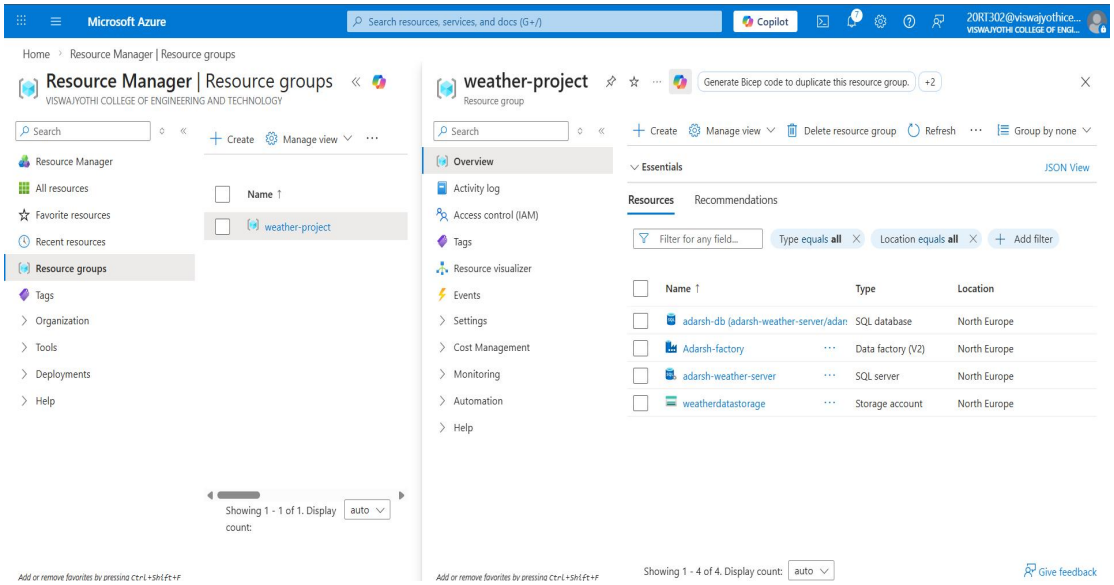
2. Project Objectives

- Learn how cloud data pipelines work in Azure.
- Understand Blob Storage, Data Factory, Managed Identities, and SQL Database.
- Build a full pipeline from Python API -> Azure Blob -> SQL -> Power BI.
- Visualize data effectively in Power BI.
- Compare Azure with AWS from a personal experience.

3. Architecture & Technologies Used

Component	Purpose	Tool/Service
Data Source	Fetch live weather data	Open-Meteo API (Python)
Data Ingestion	Extract and clean CSV	Python (Pandas)
Storage	Raw data storage	Azure Blob Storage
Integration	Move data to database	Azure Data Factory
Database	Structured storage	Azure SQL Database
Visualization	Dashboard	Power BI

Resource Group Overview:



4. Data Extraction (Python)

I wrote a Python script that pulled live weather data for Dublin, London, New York, and Dubai. The script cleaned the data and stored it as a CSV with timestamp, city, temperature, and windspeed.

Data Source (API):

- Open-Meteo API: https://api.open-meteo.com/
- Cities queried: Dublin, London, New York, Dubai
- Endpoint used: Real-time weather data in JSON format

Example of csv received:

Timestamp	City	Temperature	Windspeed
11-03-2026 11:27:00	Dublin	9.8	27
11-03-2026 11:27:00	London	11.2	18

This step taught me the basics of API integration and preprocessing before sending data to the cloud.

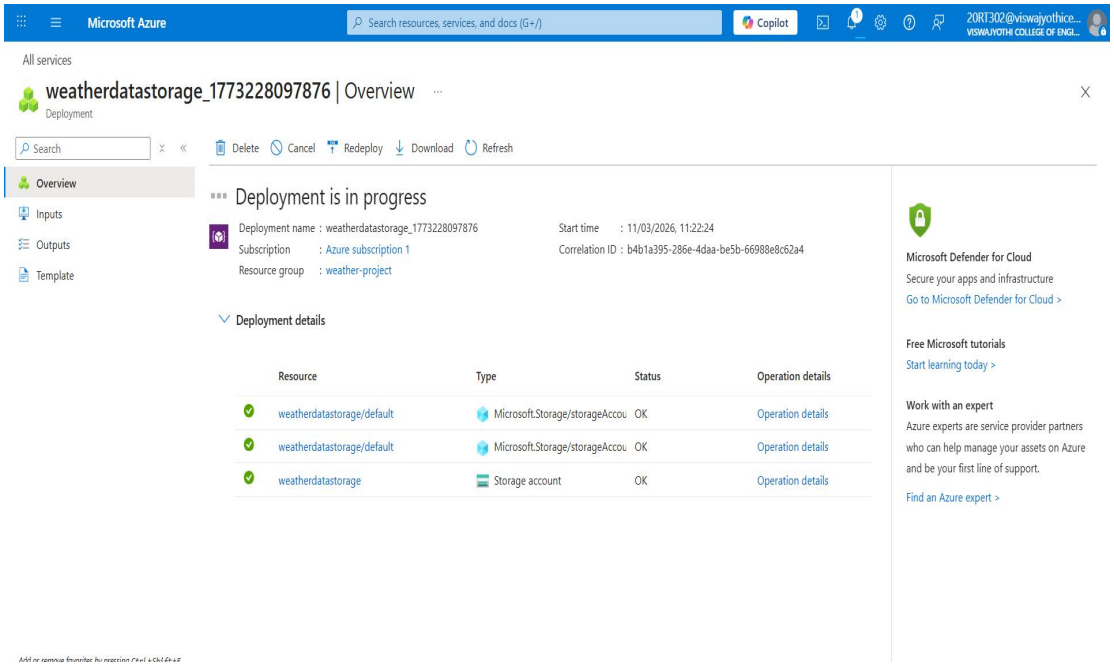
Python Script for Weather Data Extraction:

```python
# Setup folder structure
output_dir = "raw"
os.makedirs(output_dir, exist_ok=True)

# API Configuration
cities = {
    "Dublin": (53.3498, -6.2603),
    "London": (51.5072, -0.1276),
    "New York": (40.7128, -74.0060),
    "Dubai": (25.2048, 55.2708)
}

rows = []

# Fetch Data
for city, (lat, lon) in cities.items():
    url = f"https://api.open-meteo.com/v1/forecast?latitude={lat}&longitude={lon}&current_weather=true"
    try:
        data = requests.get(url).json()
        rows.append({
            "timestamp": datetime.now().strftime('%Y-%m-%d %H:%M:%S'),
            "city": city,
            "temperature": data["current_weather"]["temperature"],
            "windspeed": data["current_weather"]["windspeed"]
        })
    except Exception as e:
        print(f"Failed to fetch {city}: {e}")

# Create DataFrame and Save
df = pd.DataFrame(rows)
filename = f"weather_{datetime.now().strftime('%Y%m%d_%H%M%S')}.csv"
file_path = os.path.join(output_dir, filename)

df.to_csv(file_path, index=False)
print(f"Successfully saved: {file_path}")
```

5. Cloud Storage (Azure Blob Storage)

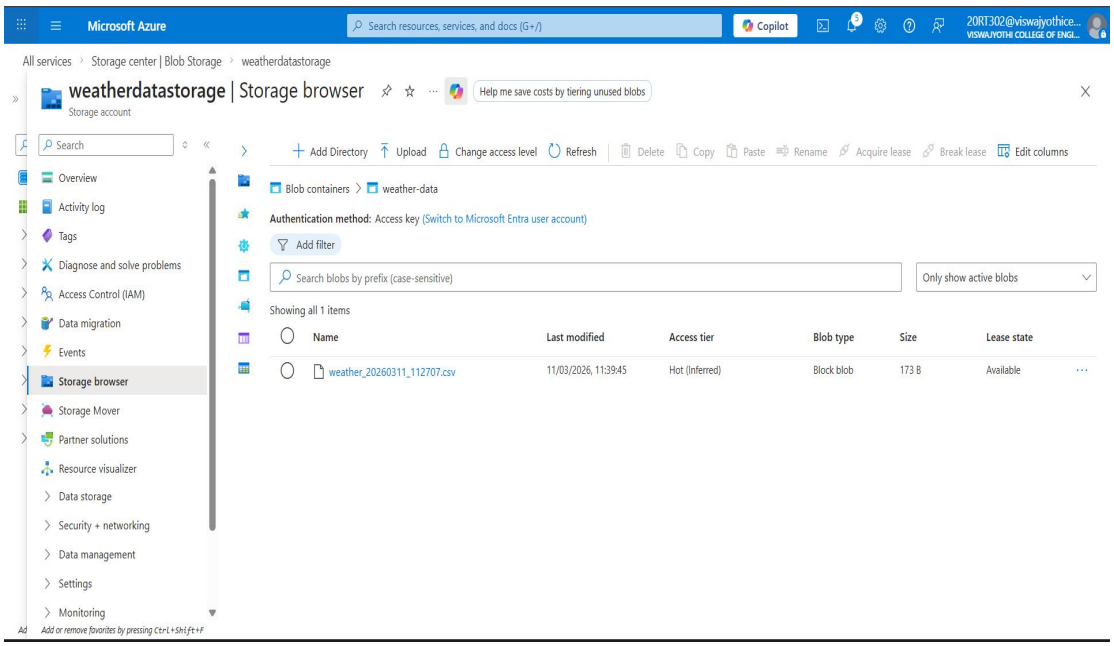
I created a Storage Account and a container to act as my data lake.

Azure Blob:



The CSV file was uploaded here. In Azure, containers are like folders for files, but actual folders don't exist-the CSV sits at the container root.

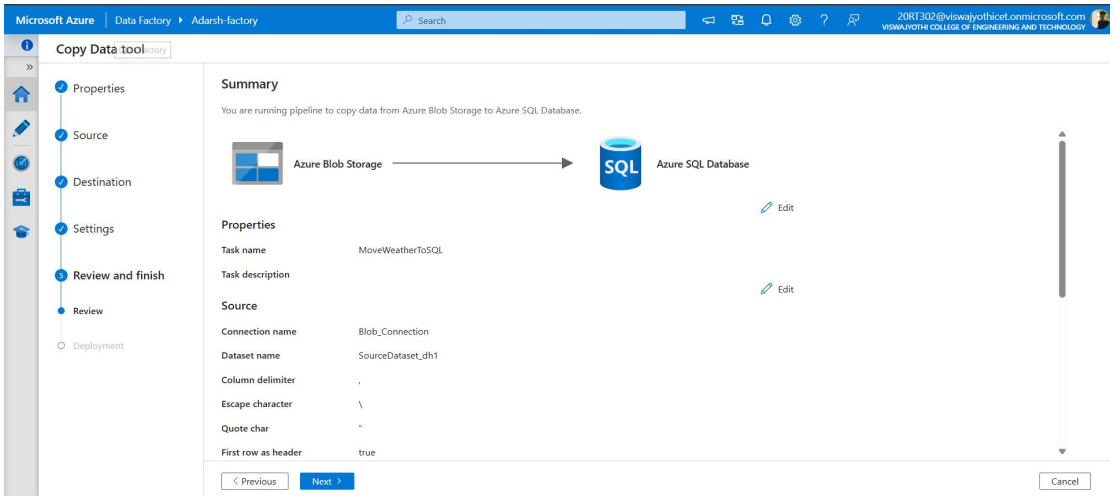
Azure Container:



6. Data Integration (Azure Data Factory)

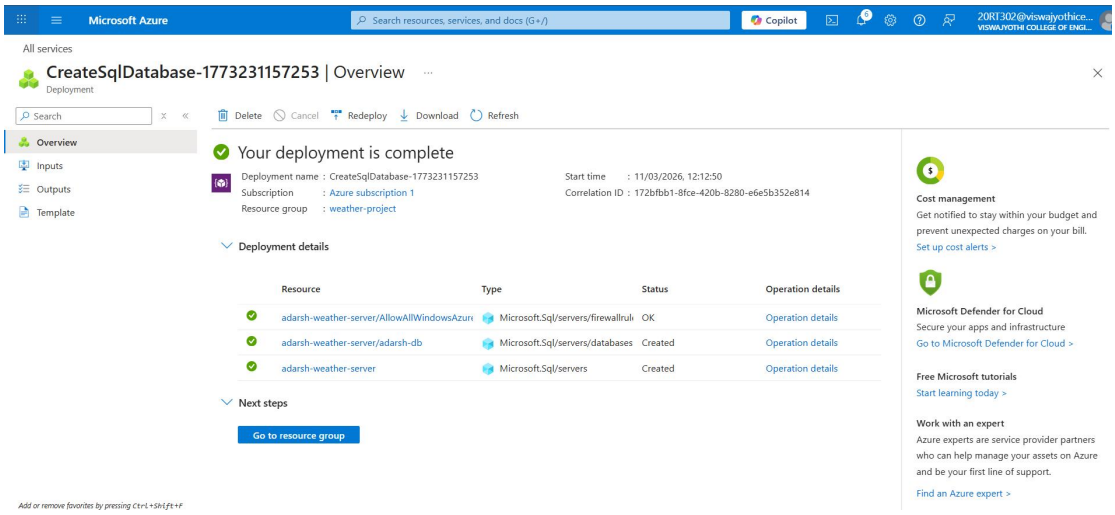
Using Data Factory, I built a pipeline to move the CSV from Blob Storage into my SQL Database. Linked Services were used to connect the storage account to the database securely, with Managed Identities ensuring credentials weren't exposed.

Data Factory:



7. Data Warehousing (Azure SQL Database)

I created a SQL-Database and a table to store the weather data with proper data types.

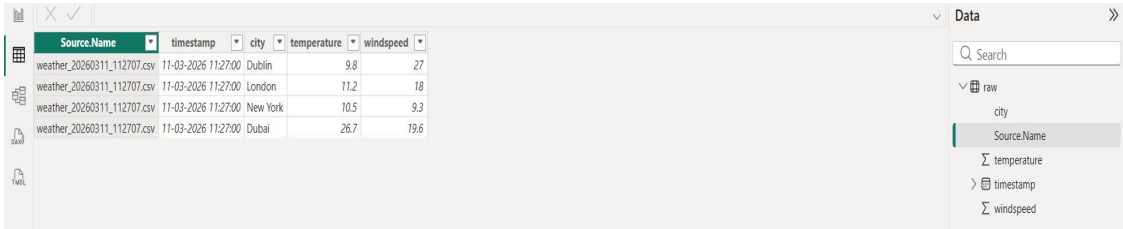



```
CREATE TABLE WeatherData (
   timestamp DATETIME,
   city NVARCHAR(50),
   temperature FLOAT,
   windspeed FLOAT
);
```

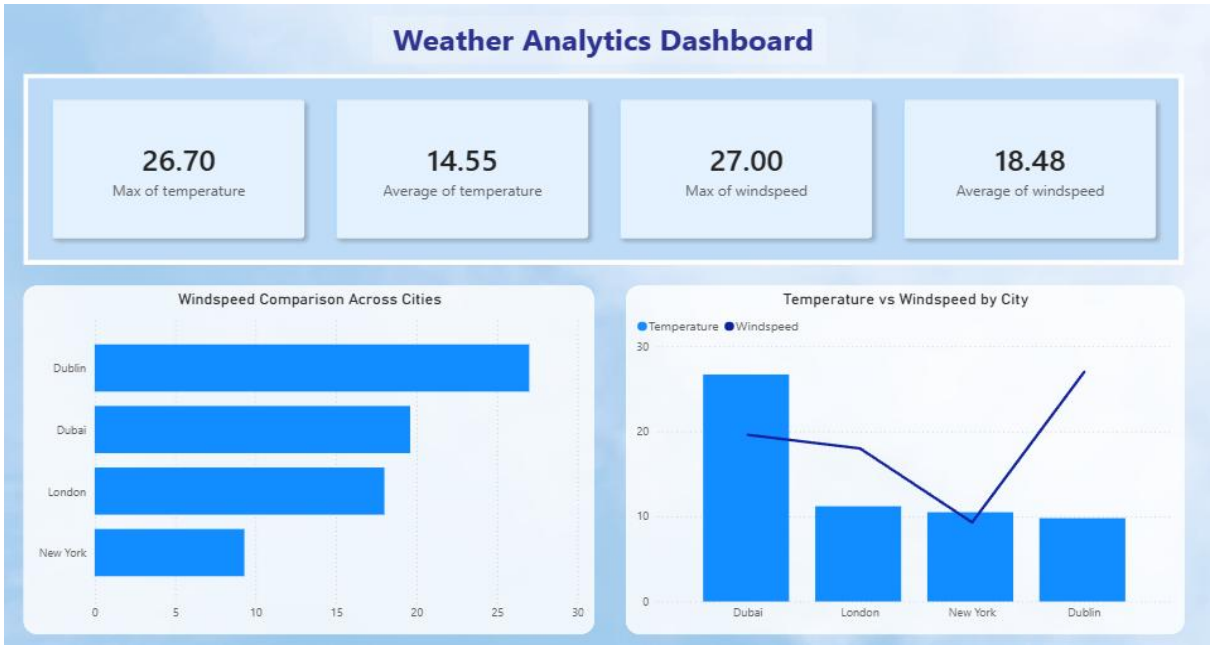
Data Factory populated this table directly from the CSV.

8. Data Visualization (Power BI)

I connected Power BI to the SQL Database and created a dashboard:



Source.Name	timestamp	city	temperature	windspeed
weather_20260311_112707.csv	11-03-2026 11:27:00	Dublin	9.8	27
weather_20260311_112707.csv	11-03-2026 11:27:00	London	11.2	18
weather_20260311_112707.csv	11-03-2026 11:27:00	New York	10.5	9.3
weather_20260311_112707.csv	11-03-2026 11:27:00	Dubai	26.7	19.6



9. Challenges

- Azure has more overhead compared to AWS (S3 + Lambda is faster for small projects).
- Managed Identities and Linked Services add security but increase complexity.
- Strict Schema Mapping in Azure Data Factory.

10. Future Improvements

- Automate data extraction with Azure Functions
- Store historical weather data for trends

11. Conclusion

This project demonstrates a full Azure pipeline:

Python -> Blob Storage -> Data Factory -> SQL Database -> Power BI

This pipeline demonstrates how quickly raw, external data can be securely ingested, transformed, and visualized using Azure's ecosystem. Mastering tools like Data Factory and Azure SQL not only expanded my cloud engineering capabilities, but pairing this with my AWS experience allows me to approach architectural challenges with a highly versatile, multi-cloud mindset.